# INGENIERÍA DE SOFTWARE

## Video 1.
## Diagrama de Clases desde cero

Enlace del video

Este tutorial explica progresivamente cómo construir e interpretar un diagrama de clases UML. Comienza con la representación de una clase mediante nombre, atributos y métodos; luego incorpora visibilidad y desarrolla las principales relaciones entre clases, como herencia, asociación, agregación y composición. También explica multiplicidades mediante ejemplos concretos y culmina con un ejemplo aplicado que permite observar cómo los distintos elementos se integran en un modelo estructural. Por su claridad visual y secuencia didáctica, es probablemente la alternativa en español más parecida al primer video de Lucid que seleccionamos originalmente.

**Pregunta para reflexionar:**

> Si dos analistas reciben los mismos requisitos de un sistema, ¿por qué podrían construir diagramas de clases diferentes y cómo determinarías cuál representa mejor el problema real?

## Video 2.
## Diagrama de clases

Enlace del video

Este recurso académico de la Universitat Politècnica de València introduce el modelado conceptual mediante diagramas de clases UML. Explica el concepto de clase y su representación mediante nombre, atributos y operaciones, así como la visibilidad de sus elementos. Posteriormente desarrolla asociaciones, multiplicidades, agregación, composición y relaciones de especialización/generalización. Incluye ejemplos que permiten comprender que el diagrama de clases representa la estructura estática del sistema y no simplemente un conjunto de símbolos gráficos.

**Pregunta para reflexionar:**

> ¿Qué consecuencias podría tener para el desarrollo del software construir un diagrama de clases técnicamente correcto en su notación, pero que no represente adecuadamente las necesidades y reglas del negocio?

## Video 3.
## https://youtu.be/GZ2lQ9GrlDQ

Enlace del video

El video profundiza en la interpretación y utilización de las relaciones UML dentro de un diagrama de clases. Aborda asociación, multiplicidad, agregación, composición, herencia y polimorfismo, enfatizando que cada relación expresa decisiones concretas acerca de la estructura y las reglas del sistema. Mediante ejemplos aplicados, invita a interpretar el significado del modelo y a justificar por qué determinadas clases deben relacionarse de una forma específica, evitando utilizar la notación UML de manera meramente gráfica o mecánica.

**Pregunta para reflexionar:**

> ¿Qué problemas podrían aparecer en un sistema si elegimos incorrectamente las relaciones entre sus clases, aunque todas las clases necesarias hayan sido identificadas?